



cover

Projeto do Arthur

Desenvolvimento Back-end - Experiência Prática 3



2 Identificação do aluno

Projeto do Arthur
Desenvolvimento Back-end -
Experiência Prática 3

AS

**Arthur Fernando Rodrigues Fonseca CRUZ_EAD_Cst em Análise e
Desenvolvimento de Sistemas_3A_20262**



3 Seleção da aplicação web

Definição da plataforma de estudo

A aplicação web escolhida para a investigação estrutural foi o **YouTube**, utilizando sua versão web acessada pelo navegador. A escolha foi feita porque a plataforma apresenta diversas funcionalidades que geram interações entre o cliente e o servidor em tempo real. Entre elas estão o carregamento da página inicial, a pesquisa de vídeos, o carregamento de informações dos vídeos e canais, além da reprodução de conteúdos. Essas ações permitem observar diferentes requisições HTTP realizadas pelo navegador durante a utilização da plataforma. Dessa forma, o YouTube apresenta um cenário adequado para analisar a comunicação entre o front-end e o back-end, identificar requisições e métodos HTTP e observar as respostas retornadas pelo servidor por meio das ferramentas de desenvolvedor do navegador.



Explicação da arquitetura cliente-servidor

No YouTube, a arquitetura cliente-servidor é dividida entre o front-end, executado no navegador do usuário, e o back-end, responsável pelo processamento das informações e pela disponibilização dos serviços da plataforma. O cliente utiliza tecnologias como HTML, CSS e JavaScript para apresentar a interface e permitir que o usuário realize ações como pesquisar vídeos, selecionar conteúdos, navegar pelos canais e interagir com os recursos disponíveis na plataforma. O servidor e suas APIs são responsáveis por receber as requisições realizadas pelo cliente, processar as informações necessárias e devolver as respostas para que a interface possa apresentar os dados corretamente. Nesse processo, o back-end pode trabalhar com dados relacionados aos vídeos, canais, pesquisas, autenticação e demais recursos necessários ao funcionamento da plataforma. A comunicação entre cliente e servidor ocorre por meio de requisições e respostas HTTP. Dessa forma, o cliente concentra a apresentação da interface e a interação com o usuário, enquanto o servidor concentra o processamento das informações e a disponibilização dos dados e serviços solicitados. Essa divisão permite que o navegador solicite informações e receba as respostas necessárias para atualizar e apresentar o conteúdo do YouTube.



5 Utilização das ferramentas de inspeção

Primeira requisição observada

A primeira requisição Fetch/XHR identificada foi o endpoint search, acionado após a realização de uma pesquisa no YouTube. A requisição apresentou código de status HTTP 200, indicando que a comunicação com o servidor foi concluída com sucesso. Na resposta, foram retornados dados relacionados aos resultados da busca, como títulos e informações das imagens dos vídeos.



6 Mapeamento das requisições HTTP (1/2)

URL ou endpoint da requisição e método HTTP (ex: GET, POST)

- /complete/search - GET
- /youtubei/v1/get_watch - POST
- /api/stats/watchtime - POST
- /videoplayback - POST
- /youtubei/v1/log_event?alt=json - POST

Status Code (ex: 200, 404)

- 200
- 200
- 204
- 200
- 200

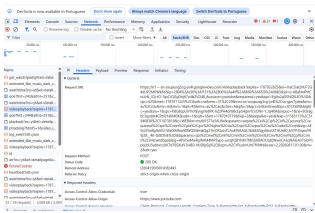
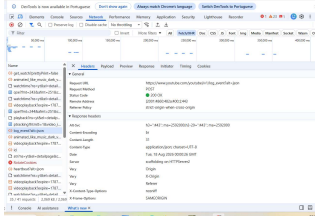
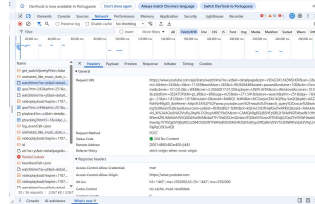
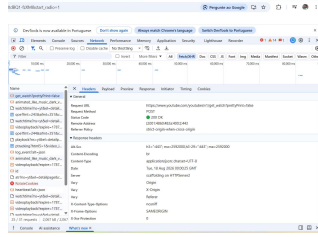
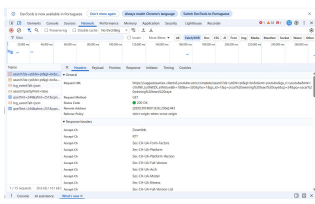
Função da requisição na aplicação

- Realizar a pesquisa no YouTube e retornar os resultados correspondentes ao termo pesquisado pelo usuário.
- Carregar os dados necessários para exibir o vídeo selecionado pelo usuário.
- Registrar informações relacionadas ao tempo de reprodução e ao estado de visualização do vídeo pelo usuário
- Responsável por solicitar os dados necessários para a reprodução do vídeo selecionado pelo usuário.
- Registrar eventos e informações de interação do usuário durante a utilização do YouTube.



7 Mapeamento das requisições HTTP (2/2)

Upload dos Prints DevTools





Análise de Método e Status Code

A requisição analisada foi a de busca do YouTube, realizada pelo endpoint `/complete/search`. O método HTTP utilizado foi o GET, pois essa requisição tem como objetivo consultar e retornar informações relacionadas ao termo pesquisado pelo usuário, sem criar, alterar ou excluir dados no servidor. Dessa forma, o GET é adequado para essa finalidade. O Status Code 200 (OK) indica que a requisição foi processada com sucesso pelo servidor e que os dados da pesquisa foram retornados corretamente para o cliente. Nesse contexto, o navegador consegue utilizar essas informações para apresentar os resultados da busca na interface do YouTube.

Interpretação do Payload JSON

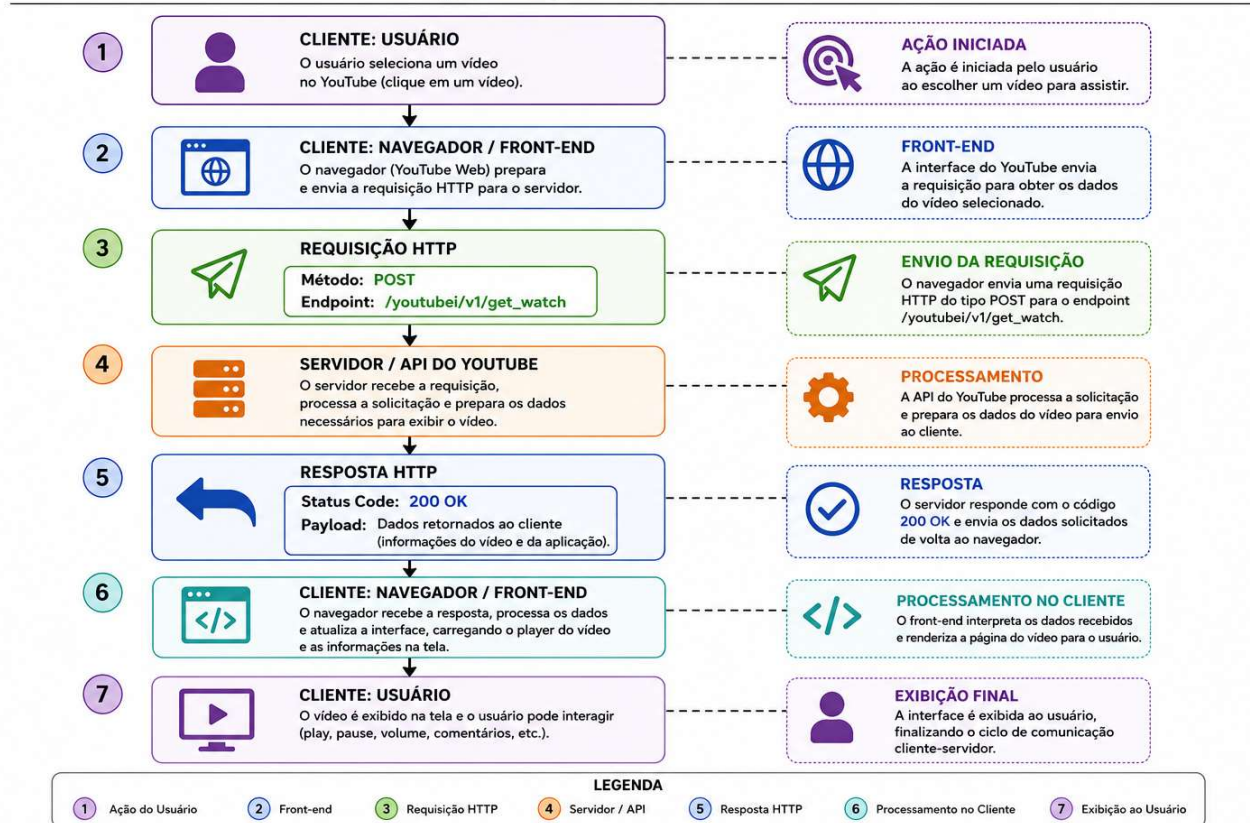
A API de pesquisa do YouTube é responsável por processar o termo pesquisado pelo usuário e retornar sugestões relacionadas à busca. No exemplo analisado, o termo utilizado foi "oscar winning tears raye". A resposta apresenta dados em formato semelhante a JSON, contendo o termo pesquisado e sugestões relacionadas, como "oscar winning tears raye" e "oscar winning tears raye alicia keys". Um trecho representativo da resposta é: "oscar winning tears raye", [{"oscar winning tears raye", 35, [39]}, ["oscar winning tears raye alicia keys", ...]] Esses dados são utilizados pelo front-end para apresentar sugestões de pesquisa e auxiliar o usuário durante a busca. Dessa forma, a comunicação entre o navegador e a API permite que os resultados sejam obtidos dinamicamente sem a necessidade de recarregar toda a página.



Upload do fluxograma

FLUXOGRAMA DE COMUNICAÇÃO CLIENTE-SERVIDOR

Exemplo: Carregamento de um vídeo no YouTube



Descrição do fluxo visual

O ciclo de vida da requisição inicia-se com a interação do usuário no YouTube Web, ao selecionar um vídeo para assistir. Nesse momento, o navegador atua como cliente e o front-end prepara a comunicação com o servidor, enviando uma requisição HTTP para obter os dados necessários para a exibição do conteúdo selecionado. A requisição analisada utiliza o método POST e o endpoint `/youtubei/v1/get_watch`. A chamada é realizada para solicitar ao servidor as informações necessárias relacionadas ao vídeo selecionado. O navegador envia essa requisição ao servidor/API do YouTube, que recebe os dados e realiza o processamento necessário para atender à solicitação. Após o processamento, o servidor devolve uma resposta HTTP com o código de status 200 OK, indicando que a requisição foi processada com sucesso. A resposta contém dados que são encaminhados de volta ao cliente para que possam ser utilizados pela aplicação. Ao receber a resposta, o navegador processa os dados retornados e atualiza a interface do YouTube, permitindo a apresentação do vídeo e das respectivas informações ao usuário. Dessa forma, a resposta HTTP encerra o ciclo específico analisado: o usuário inicia a ação, o cliente realiza a requisição, o servidor processa a solicitação, devolve uma resposta 200 OK e o cliente utiliza os dados recebidos para atualizar a interface.



Parecer técnico final

A análise realizada no YouTube Web permitiu observar, na prática, o funcionamento da arquitetura cliente-servidor e a comunicação entre o front-end e o back-end por meio de requisições HTTP. O navegador atua como cliente, responsável por receber as interações do usuário, solicitar informações e apresentar os dados retornados pelo servidor. Já o servidor/API processa as solicitações recebidas e fornece as informações necessárias para que a aplicação funcione corretamente. Durante a investigação com o DevTools, foi possível identificar diferentes requisições HTTP, seus respectivos métodos, endpoints e códigos de status. Um exemplo analisado foi a requisição POST para o endpoint `/youtubei/v1/get_watch`, que retornou o status 200 OK, indicando que a solicitação foi processada com sucesso. Também foi possível observar respostas contendo dados estruturados, demonstrando a importância das APIs na comunicação entre as diferentes partes da aplicação. A utilização dessa arquitetura permite uma clara separação de responsabilidades. O cliente concentra a apresentação e a interação com o usuário, enquanto o servidor fica responsável pelo processamento das solicitações e pelo fornecimento dos dados. Essa divisão contribui para a organização, manutenção e evolução de aplicações web modernas. Como desenvolvedor back-end em formação, compreender a anatomia das APIs é fundamental para construir e manter sistemas eficientes. Conhecer os métodos HTTP, os códigos de status, os endpoints e os formatos de dados, como o JSON, permite compreender não apenas como uma aplicação realiza suas operações, mas também como diagnosticar problemas e estabelecer uma comunicação adequada entre cliente e servidor. Portanto, a experiência demonstrou que o conhecimento da comunicação HTTP e da arquitetura de APIs constitui uma base essencial para o desenvolvimento de sistemas computacionais modernos, permitindo que o profissional compreenda o funcionamento dos serviços por trás das interfaces e desenvolva soluções mais organizadas, confiáveis e eficientes.



Checklist de qualidade

- Definição clara da aplicação web de estudo e documentação da sua arquitetura base.
- Captura bem-sucedida de requisições HTTP utilizando as ferramentas de inspeção do navegador (DevTools - aba Network).
- Mapeamento e registro técnico de pelo menos 5 requisições distintas.
- Análise aprofundada dos métodos HTTP, códigos de status e payloads JSON observados.
- Construção correta de um fluxograma visual representando o ciclo de comunicação cliente-servidor.
- Redação do parecer técnico fundamentado sobre a importância da arquitetura API REST.



Carregando gráfico...

Reflexão sobre a aprendizagem

Durante esta experiência prática, considero que tive um bom desenvolvimento das competências propostas, principalmente na compreensão da comunicação entre cliente e servidor e na identificação do funcionamento de APIs por meio de requisições HTTP. A investigação do YouTube utilizando as ferramentas DevTools permitiu que eu observasse na prática conceitos que anteriormente conhecia principalmente de forma teórica, como métodos HTTP, endpoints, códigos de status e respostas estruturadas. Como ponto forte, destaco minha capacidade de acompanhar o processo de investigação, identificar requisições relevantes e relacioná-las às ações realizadas pelo usuário na aplicação. Também consegui organizar as informações coletadas e representá-las por meio de um fluxograma, compreendendo melhor o ciclo entre cliente, requisição, servidor e resposta. Como oportunidade de melhoria, considero importante aprofundar meus conhecimentos sobre análise de tráfego de rede, APIs e arquitetura de sistemas, principalmente para interpretar com maior autonomia as informações apresentadas pelas ferramentas de desenvolvimento. Também pretendo desenvolver uma maior familiaridade com diferentes tipos de requisições e estruturas de dados utilizadas em aplicações reais. Os conhecimentos adquiridos nesta experiência contribuem diretamente para minha formação profissional em desenvolvimento back-end, pois compreender como uma aplicação se comunica internamente é fundamental para desenvolver, testar, documentar e solucionar problemas em sistemas web. A atividade também reforçou a importância de analisar não apenas o código, mas todo o fluxo de comunicação que permite que uma aplicação funcione de maneira organizada e eficiente.